

IDS, NOMENCLATURE & THE NUMBERING SYSTEM

What every id means, and when it's *born*.

A working reference for the identifier system: the SIF creator id, the Post (P) and Collab (C) numbers, the four-part nomenclature, and the exact rules that mint them – for both the live project and the historic archive.

SIF-n

ONE FROZEN ID
PER CREATOR

P + C

BOTH MINTED
AT ONBOARDING

11,252

HISTORIC
DELIVERABLES

U

LIVE NUMBERING
CONTINUES HISTORIC

01 How an id is born

Reach-out creates the row but mints nothing. The Post (P) and Collab (C) ids are both born at onboarding, the moment an order attaches.

A reach-out saves a row, but it carries no name: `post_id`, `collab_id` and the nomenclature are all NULL, and the row is addressed only by its internal bigserial `id` – the board shows it as “Pending”. The identity is granted at onboarding, when an order is attached and `mint_onboarding_block` reserves the collab and a contiguous block of post numbers in one advisory-locked call.

i Reach Out Row saved, identified by bigserial id. The SIF is minted here if the creator is new. <code>id</code> ✓ SIF ✓ <code>post_id</code> <code>collab_id</code>	ii Onboarding An order attaches. <code>mint_onboarding_block</code> reserves the C and the P-block at once. <code>post_id</code> ✓ <code>collab_id</code> ✓ nomenclature ✓	iii Posting The live link, date and final status land. The ids never change again. <code>post_link</code> · date status → Posted	iv Continuity The mint reads <code>posts ∪ historic</code> , so a returning legacy creator keeps numbering. <code>maxP</code> , <code>maxC</code> over both tables
--	---	--	---

ANATOMY – SIF-1007, A CREATOR WITH HISTORIC C1, C2 / P1, P2

SIF-1007-P3

`post_id` – one per reel or static post. The P climbs per creator across every collab; here P3 because P1–P2 already exist. No -C lives here.

SIF-1007-C3

`collab_id` – one per order. C3 because the creator already had C1, C2. The three new deliverables P3/P4/P5 all share `collab_id` SIF-1007-C3.

One SIF, frozen. Every creator holds one id of the form `SIF-n`, the only global linear counter besides campaigns. It is minted once, at the first reach-out, and never re-issued – Meta ad-name matching depends on it staying stable.

P and C are separate columns. A `post_id` never contains a -C, and a `collab_id` never contains a -P. The P counts deliverables; the C counts orders. They share only the SIF root.

Per-creator, not global. Both P and C climb within a creator. Every creator has their own P1 and C1; the numbers say nothing about volume across the platform.

Historic continuity. The mint takes `max(P)` and `max(C)` over `posts ∪ historic_posts`, so a legacy creator who ended at P50/C5 gets P51 and C6 next – never a collision with their own past.

A reach-out has a contact, not yet a name. Onboarding gives it both an order and an identity, in the same locked breath.

02 The SIF, the Post id & the creator key

The creator root, the deliverable number that hangs off it, and the numeric id that survives a handle change.

Every other id is built on the SIF. The Post id adds a per-creator deliverable counter; until onboarding stamps it, the row lives only by its database serial; and the `profile_id` keeps a creator recognisable even when their @handle changes.

- a **Creator SIF id** `SIF-{n}` `both`
- Each creator gets one unique `SIF-n` where n is the next integer after the highest `sif_number` in the registry. Minted at the first reach-out, reused forever, frozen for the life of the creator.
- ```
submit_reachout: lock reachout-user:{username} → if username unknown, n = max(sif_number)+1,
insert creator; else reuse inf_id. inf_id = 'SIF-'||sif_number.
```
- `SIF-1` `SIF-7829` `frozen · Meta-matched`
- 
- b **Post id (deliverable)** `{sif}-P{post_number}` `both`
- One per reel or static post. Combines the SIF and a per-creator post counter. There is *no -C suffix* – collab grouping lives in the separate `collab_id` column. `post_id` and `post_id_short` hold the same value today.
- ```
mint_onboarding_block: post_id_base = inf_id||'-P'||(maxP+1); stamped on the row only on the
first onboard (parent.post_id was NULL).
```
- `SIF-1-P1` `SIF-1-P2` `NULL until onboarded`
-
- c **Post number (the “P”)** `integer, per-creator` `both`
- The counter inside the P. It climbs per creator across *all* their collabs and continues over the historic archive – not global, not per-campaign. Children of a multi-deliverable collab take consecutive numbers.
- ```
start = max(post_number) over (posts u historic_posts WHERE inf_id) + 1. Children take start+1,
start+2... with no re-query (that would race the reservation).
```
- `C2 of 3 posts → P2,P3,P4` `stories don't increment P`
- 
- d **Reach-out row identity** `bigserial id` `new`
- A reach-out row has no `post_id` yet, so it is addressed by the internal `posts.id` – rendered “Pending” on the board. Once onboarded, the `post_id` is stamped and becomes the external key, while `id` stays the true primary key.
- UNIQUE(`post_id`) permits many NULLs (NULLs are distinct in Postgres). `editReachOut` / test-stamping key on `id` while `post_id` is NULL.
- `id=143, post_id=NULL` `→ onboard → SIF-1-P1`
- 
- e **profile\_id — identity across handle changes** `Instagram numeric id` `both`
- The creator’s Instagram numeric account id (legacy `ig_id`), stored on the creator beside the SIF. Handles change; the numeric id does not – so a returning creator under a new @handle is re-identified by `profile_id`, keeping the same SIF with no renumber.
- Copied from the Meta / historic lookup onto `creators.profile_id` and `historic_posts.profile_id`. ~95% backfilled on the historic registry; NULL where enrichment failed (~5%).
- `8489263759` `survives @handle change`

# 03 The Collab id

*One order is one collab. Every deliverable of that order shares a single collab id, minted idempotently from the order number.*

A collab represents one agreement. Re-submitting the same order reuses the same collab id; a different order mints the next one. The -C suffix lives only here.

- a **Collab id** `{sif}-C{collab_number}` both
- All deliverables of one order – a single reel, or three reels plus two posts – carry the same `collab_id`. Re-onboarding the same order is idempotent: it reuses the stored id, never minting a duplicate.
- `mint_onboarding_block`: normalize `order_id`; if it already maps to a collab for this creator, reuse that `collab_number`; else `max(collab_number over posts ∪ historic) + 1`. `collab_id = inf_id||'-C'||n`.
- SIF-1-C1 (order #123)   SIF-1-C2 (order #456)   re-onboard #123 → C1
- b **Collab number (the “C”)** `integer ≥ 1, per-creator` both
- Which collaboration episode this is for the creator. SIF-1’s first order is C1, second C2. A returning legacy creator continues from their historic max, so a first *live* collab can be C6.
- Same derivation as `collab_id`. CHECK constraint `collab_number ≥ 1`. Two deliverables sharing one order share one `collab_number`.
- legacy C1-C5 → next C6   same order = same C
- c **Deliverable index & type** `index 1,2,3... · reel | post` new
- Within a collab, deliverables are ordered: parent = 1, children 2, 3... The type marks reel vs static post. Grouping is by `collab_id`; the index is ordering and historic compatibility only.
- Parent `deliverable_role='single' | 'parent'`, index 1; children index 2+. **Stories are counted but never spawn a typed row** – the parent’s stories are set to 0 at onboarding.
- parent reel → 1   child post → 2   type ∈ {reel, post}
- d **Order-id normalization** `lower(btrim(strip #))` both
- Before deciding whether an order already owns a collab, the order id is canonicalized: a leading # is stripped, whitespace trimmed, the value lowercased. Matching ids map to the same collab.
- `lower(btrim(regexp_replace(order_id, '^#+', '')))`. Leading zeros are NOT stripped – 0123 ≠ 123. An empty order id always mints a fresh collab.
- #123 → 123   ' ABC ' → abc

*The collab is the order, made nameable. Everything a creator delivers for one order lives under one -C.*



# 04 The nomenclature

*The human-readable audit label, stamped at onboarding. Four dash-joined segments – and the post id inside it never carries a -C.*

Nomenclature is the readable name written onto every deliverable when it is onboarded.

Reach-out rows have none; re-onboarding preserves an existing value.

THE FORMAT, READ LEFT TO RIGHT

**SIF-15-P42** – johndoe – Reel – 2026-06-15

POST ID · NO -C USERNAME CONTENT TYPE REACH-OUT DATE

## a Format string `{post_id}-{username}-{content_type}-{reach_out_date}`

Four segments joined by dashes. The first is the deliverable `post_id` ( `SIF-N-P{n}`, no -C); the -C lives only on the separate `collab_id` column, never inside the name.

`buildLegacyNomenclature(postId, username, contentType, date) →` the four-part string; returns NULL if `post_id`, `username` or `content_type` is missing. Applied to the parent and every child at onboarding.

SIF-15-P42-johndoe-Reel-2026-06-15

## b Date segment `YYYY-MM-DD · Asia/Kolkata`

An ISO calendar date in Indian Standard Time, capturing when *outreach* happened – not the onboarding moment. The same reach-out date is reused across a creator's later collabs.

`toLocaleDateString('en-CA', {timeZone: 'Asia/Kolkata'})` – en-CA yields YYYY-MM-DD. Falls back to today if `reach_out_date` is absent. No time component.

2026-06-15 reach-out date, reused

## c Username & content-type segments `lowercase handle · free-text code`

The username is the Instagram handle without the @, lowercased and frozen once the row leaves Reach Out. The content type is an operator-entered label (UGC, Reels, VRP, IFADs...) with no fixed enum – the allowed set is whatever already exists in the data.

Username extracted via `instagram.com/@?([A-Za-z0-9._]+)` then lowercased. Content type editable only while in Reach Out (decision D7), which rebuilds the nomenclature.

johndoe UGC VRP IFADs

*A name is built only once the post id exists. So the name, like the id, is a thing of onboarding – never of reach-out.*

# 05 New & Historic

*Same grammar, different provenance. The two are welded by deliberate numbering continuity.*

Live ids are minted transactionally at runtime; historic ids were generated once, by SQL window functions, when the legacy sheet was migrated into `historic_posts`. The read grammar is identical; only the write provenance differs.

| ASPECT          | NEW – LIVE POSTS                                  | HISTORIC – ARCHIVE                                                                     |
|-----------------|---------------------------------------------------|----------------------------------------------------------------------------------------|
| How minted      | Runtime, under advisory locks                     | Once, by window functions at migration                                                 |
| SIF id          | At reach-out (submit_reachout)                    | Frozen from the original legacy sheet                                                  |
| P & C minted    | At onboarding (mint_onboarding_block)             | <code>ROW_NUMBER()</code> / <code>DENSE_RANK()</code>                                  |
| post_number     | Per-creator; continues over union                 | <code>ROW_NUMBER()</code> PARTITION BY <code>username</code>                           |
| collab_number   | Reuse-on-order else max+1                         | <code>DENSE_RANK()</code> over distinct <code>order_id</code>                          |
| Reach-out rows  | NULL <code>post_id</code> – keyed by bigserial id | Carry a migrated <code>post_number</code> ; NULL <code>collab</code>                   |
| workflow_status | Full 12-value lifecycle                           | Posted / On Board / Reach Out only                                                     |
| Archive columns | –                                                 | <code>source_cleaned_id</code> , <code>profile_id</code> , <code>legacy_post_id</code> |
| creator_type    | <code>new_creator</code>                          | <code>historic_creator</code>                                                          |

*The mint reads both tables. A returning legacy creator who ended at P50 / C5 gets P51 and C6 next — the archive and the live system share one continuous count.*

## Historic derivation

`post_id` = `SIF-{sif}-P{ROW_NUMBER() PARTITION BY username}`. `collab_number` = `DENSE_RANK()` over distinct `order_id` (NULL for reach-out rows). `deliverable_index` = row number within the collab. Status derived purely from `post_link` / `order_id` presence. Every archive row keeps a `source_cleaned_id` back to its `cleaned_data` source, plus its `profile_id` and original `legacy_post_id`.

## The dedup boundary

One real profile = one canonical SIF. Where a creator appeared under several SIFs, the loser's `sif_id` was renamed `SIF_ERROR` and kept in `cleaned_data` for audit only. The clean archive is **11,252** of the **13,430** cleaned rows; the **2,178** `SIF_ERROR` markers are never promoted into `historic_posts`.

## § Campaign id — a separate global counter

Campaigns have their own namespace: `IFC{NNN}`, zero-padded to three digits from `MAX(campaign_num)+1`, advisory-lock serialized, never reset by period. The first campaign is `IFC001`.



# 06 Invariants & open edges

*The rules that keep id generation race-free and consistent – and an honest list of what is not yet clean.*

---

## INVARIANTS – ALWAYS TRUE

---

### a Advisory-lock concurrency

Reach-out locks on the username ( `reachout-user:{username}` ) so two simultaneous submits can't create duplicate creators. Onboarding locks on the creator ( `reachout-inf:{inf_id}` ) inside `mint_onboarding_block` so the P-block and C reservation are atomic. Locks release at transaction end; rollback releases before any dependent read.

### b Barter = zero, and equal split

When `collab_type='Barter'`, `commercial_amount` is forced to 0 at both the database and app layers. Otherwise the agreed total is split equally across all deliverables, so the sum across a collab reconstructs the original agreed amount.

### c Re-onboard idempotency

Re-onboarding an already-minted row never re-mints. The guard is `isFirstOnboard = parent.post_id IS NULL`; when false, the stored `post_id`, `post_number`, collab id and nomenclature are all preserved, and children are not re-spawned.

---

## OPEN EDGES – FLAGGED FOR RECONCILIATION

---

### ! Tier boundary: 300K vs 500K

The campaign form labels *Mid tier* as 50K–500K, but runtime `tierFromFollowers` uses a <300K cutoff. A 400K creator is Mid tier by the form yet Macro by the generated `category` column. Pick one boundary.

### ! collab\_type: is bare 'Paid' legal?

Production allows only **Barter** and **Barter + Paid**. The campaign form still shows a **Paid** option, most likely a legacy mislabel of Barter + Paid. Reconcile the two schemas.

### ! post\_id\_short duplicates post\_id

Both columns hold the identical value today – a remnant of the pre-2026-06-06 model where `post_id` carried a -C suffix. Treat `post_id_short` as redundant; never rely on a difference.

### ! Transient: 164 Posted-but-no-order historic rows

164 historic Posted rows have a real Instagram link but NULL `order_id` → NULL collab. The team is backfilling order ids; once done these gain proper collab ids and the anomaly clears. As of 2026-06-25, treat their collab fields as pending.

SAADAA CREATORHUB · FIELD MANUAL № 02 · IDENTITY & NAMING · V2 · 2026-06-25